Web Systems Development

Authentication & authorization

Session 21 · Tue 11/10 · Who are you? What may you do? Cookies, OAuth 2.0 and JWT, then guards and policies in Laravel

RPI

READINGS

Read before class: authentication and authorization

Priority	Reading	Sections to complete
Required	1. Laravel docs, Authentication	Authentication Quickstart; manually authenticating users; logging out
Required	2. Laravel docs, Authorization	Gates; creating and writing policies; authorizing actions with policies
Required	3. Laravel docs, Sanctum	Introduction; how Sanctum works; SPA authentication
Required	4. Laravel docs, Validation, CSRF Protection, Session	Validation Quickstart; CSRF protection (including X-XSRF-TOKEN); sessions
Required	5. Laravel docs, Rate Limiting and Eloquent: Query Scopes	Defining and attaching rate limiters; global query scopes
Required	6. Parecki, OAuth 2.0 Simplified: Authorization Code and PKCE	Read both chapters
Optional	7. OpenID Foundation, How OpenID Connect Works	Read the full overview page
Optional	8. Laravel docs, Socialite	Introduction through "Retrieving User Details"
Optional	9. Laravel docs, Passport	Introduction; Authorization Code Grant with PKCE

The titles are links. Today uses items 1 to 3 and 6; items 4 and 5 support Friday's lab and Homework 9.

Today: two questions, three technologies, two Laravel tools

PART 1

The two questions

Who are you? What may you do?

You define authentication and authorization first. Then we pin both down, and watch both happen in the pantry app.

PART 2 · AUTHENTICATION

Three technologies

How does a request prove who sent it?

Cookies, OAuth 2.0 and JWT: a definition and a worked example of each, and where each one shows up in Laravel.

PART 3 · AUTHORIZATION

Guards and policies

Who decides what a user may do?

In Laravel a guard supplies the user and a policy makes the decision. The pantry's real code, and the 401 and 403 it produces.

BUILDS ON

Homework 7: your api/ and its three users. Last week: HttpOnly and CSRF.

THEN, IN CLASS

The login drills, stepped through at the end of this deck. Start the build now.

LEADS TO

Friday 11/13: the team login lab. Homework 9: your login and one policy.

QUESTION 1 OF 2 · WRITE IT DOWN

What is authentication?

Take one minute. Write one sentence.

Then name one time today that a website or a device did it to you.



QUESTION 2 OF 2 · WRITE IT DOWN

What is authorization?

One sentence again.

Then name something a site let you see but would not let you change.



Authentication asks who; authorization asks what



AUTHENTICATION · "AUTHN"

Who are you?

Verifying that a person, device or service is who it claims to be.

EXAMPLE

Alex types an email and a password, and the server checks them.

WHEN IT FAILS

`401 Unauthorized` "I do not know who you are."



AUTHORIZATION · "AUTHZ"

What may you do?

Deciding whether an identified user is allowed to do this action to this thing.

EXAMPLE

Alex may edit the milk Alex added. Only the RA may delete it.

WHEN IT FAILS

`403 Forbidden` "I know who you are. The answer is no."

First who, then what. Authorization needs an identity to decide about, and both are decided on the server, on every request.

CHECK YOURSELF

Which question is it: who, or what?

1. You unlock your phone with your face.

2. A Google Doc opens, but the Edit button is greyed out.

3. The LMS shows your grade, but not your neighbour's.

4. A site texts you a six-digit code and waits.

5. `GET /api/user` answers **401**.

6. `DELETE /api/items/2` answers **403**.

Answers: *1 who · 2 what · 3 what · 4 who · 5 who · 6 what. The test: is the system finding out who you are, or has it already decided that and is now choosing what to allow? In 2 and 3 you were logged in the whole time.*

Both questions in one app: four requests to the pantry

1	`POST /api/login` Alex sends an email and a password	→	`200 Set-Cookie: floor-pantry-session=…` the password is checked once; a cookie comes back	WHO
2	`GET /api/user` the browser attaches the cookie by itself	→	`200 { "name": "Alex Rivera" }` the cookie is enough: no password this time	WHO
3	`POST /api/items` sent with no cookie at all	→	`401 Unauthorized` the server cannot tell who is asking	WHO
4	`DELETE /api/items/2` Alex, logged in, deletes Priya's yogurt	→	`403 Forbidden` "Only the RA can remove items from the shelf."	WHAT

The rest of today explains these four rows. Part 2: how the server knows it is Alex (rows 1 to 3). Part 3: why Alex is still refused (row 4).

PART 2 · AUTHENTICATION

How does a request prove who sent it?



HTTP forgets. Every request arrives as a stranger, so every request has to carry proof.

Every login makes two choices: who checks the user, and what the browser carries afterwards.

A login proves who you are once. Then what?

1

Send credentials

The browser posts an email and a password, over HTTPS.

2

Verify

The server hashes the typed password and compares it with the stored bcrypt hash.

3

Remember

The server records the result: this visitor is user 1, Alex.

4

Hand back proof

The response gives the browser something to show on its next request.

PREDICT Request 2 arrives one second later, and HTTP remembers nothing. What could the browser send so the server knows it is still Alex, without sending the password again?

Two families of answer. *A reference: a random ID the server looks up (a session cookie). A statement: signed data the server can verify by itself (a JWT). That is one choice. The other is who does steps 1 and 2: your own password form, or another service, with OAuth 2.0.*

Every login makes two choices

1

CHOICE 1

Who checks the user at login?



THE BASELINE

Your password form

Your server compares the typed password with a stored hash.



THE ALTERNATIVE

OAuth 2.0

A protocol: another service checks the password and vouches for the user.

2

CHOICE 2

What do later requests carry as proof?



THE BASELINE

A session cookie

A browser mechanism: it holds a random ID and is sent back automatically. The server looks the ID up.



THE ALTERNATIVE

A JWT

A token format: signed claims the server can verify without a lookup.

OAuth 2.0 is an alternative to the password, not to the cookie. A JWT is an alternative to the session ID. One option from each row makes a login; Homework 9 takes the baseline in both.

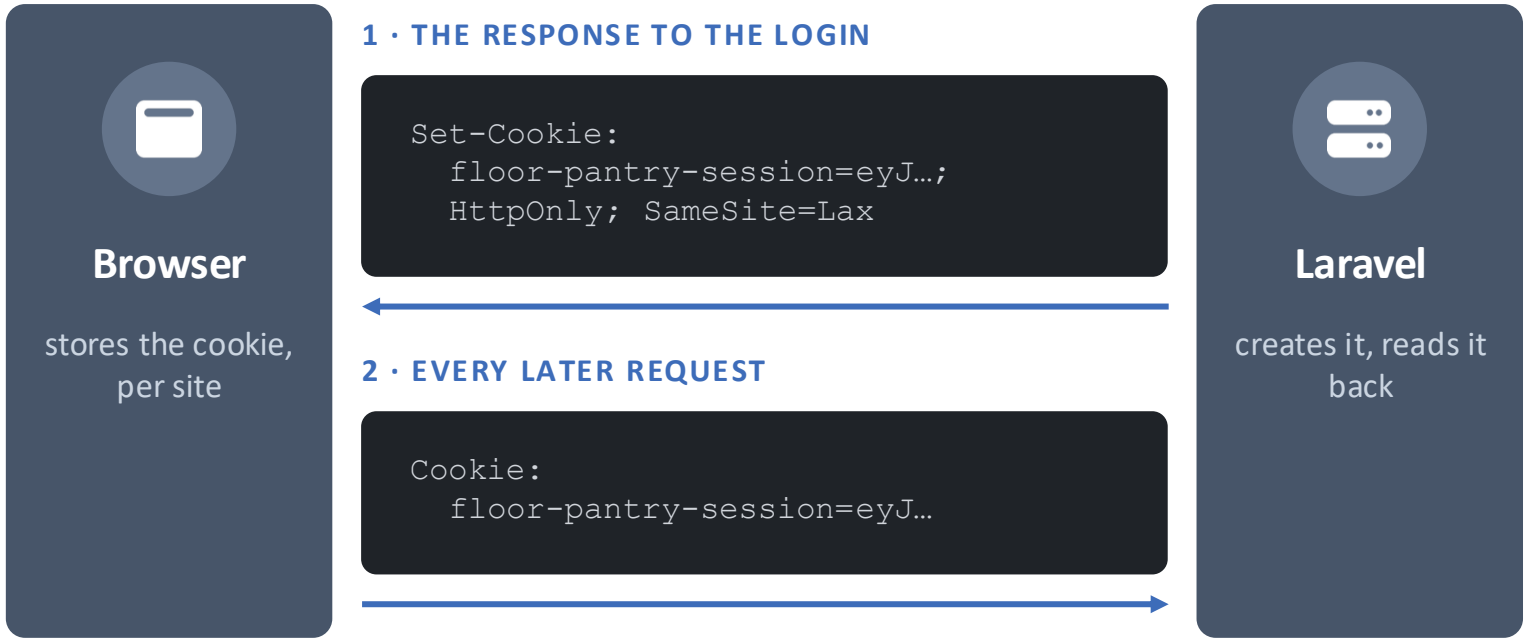
A cookie: the browser stores it and sends it back by itself

DEFINITION

A cookie is a small name=value pair that a server asks the browser to store. The browser sends it back, automatically, with every later request to that site.

WHY IT MATTERS

It is the browser's memory for a protocol that has none. Your JavaScript does nothing: that is why a login survives a page reload.



The server sets it
with a Set-Cookie response header.

The browser keeps it
in a jar for that one site only.

The browser returns it
in a Cookie request header, unasked.

The cookie holds a reference; the login lives on the server

IN THE BROWSER: A COOKIE

`floor-pantry-session = a7f3…`

A random ID. It says nothing about Alex by itself.

ON THE SERVER: THE SESSION STORE

`a7f3…  →  user_id 1`

The meaning of the ID. Looked up on every request.

IN YOUR CONTROLLER

```
$request->user()    // Alex
```

What your code sees: a User model, or nobody.

THREE FLAGS ON THE COOKIE, AND THE ATTACK EACH ONE STOPS

`HttpOnly`

JavaScript cannot read the cookie, so a script injected into your page cannot steal the login.

Last week, drill 2: XSS

`Secure`

The cookie travels only over HTTPS, so nobody on the network between can read it.

Off on localhost; on in production

`SameSite=Lax`

The browser leaves the cookie off cross-site POSTs, so another site cannot act as you.

Last week, drill 3: CSRF

Logging out means the server deletes the session. The cookie may still be in the browser, but it now points at nothing.

In class: find the cookie

1. Open the drills app at `http://localhost:8020` and log in as `alex@example.edu` / `password`.
2. Open DevTools → Application → Cookies → `http://localhost:8020`. (Firefox, Safari: Storage.)
3. Find `login-lab-session`. Which of HttpOnly, Secure and SameSite are set?
4. Click the Console tab. Run `document.cookie`. Which cookie from step 3 is missing?
5. Delete `login-lab-session` and reload My to-dos. Who are you now?

WHAT YOU SHOULD FIND

3. HttpOnly is set and SameSite is Lax. Secure is not: localhost is plain HTTP, so production turns it on.
4. Only XSRF-TOKEN prints. HttpOnly hides the session cookie from every script, which is the point.
5. A guest: you are sent to log in. The server still has the session, but the browser no longer has the reference.

Laravel names the cookie after the app: the pantry's is `floor-pantry-session`. Not running yet? Watch a neighbour, or the projector.

The baseline in Laravel: a password, then a session cookie

AuthController::login() *project/pantry-example-final, shortened*

```php
public function login(Request $request) {
    $credentials = $request->validate([
        'email'    => ['required', 'email'],
        'password' => ['required', 'string']]);
    if (! Auth::attempt($credentials)) {
        throw ValidationException::withMessages(['email' =>
            'These credentials do not match our records.']);
    }
    $request->session()->regenerate();
    return ['data' => $request->user()];
}

bootstrap/app.php: /api requests from React get a session
$middleware->statefulApi();
```

Green marks the three lines that make the cookie mean "Alex".

WHAT EACH LINE DOES

Auth::attempt finds the user by email, hashes the typed password and compares it with the stored hash. On success it writes the user's id into the session.

WHY IT WORKS

regenerate() issues a new session ID at login, so an ID planted in the browser beforehand is worthless. You never write Set-Cookie: the session middleware adds it to the response.

IN ONE LINE

attempt → session → cookie

OAuth 2.0 replaces the password form, not the cookie

PASSWORD LOGIN · THE PANTRY TODAY	
The login form is on	**your site**
The password is checked by	**your server: Auth::attempt()**
Your database stores	**a bcrypt hash of the password**

OAUTH 2.0 LOGIN · "LOG IN WITH GITHUB"	
The login form is on	**GitHub's site**
The password is checked by	**GitHub**
Your database stores	**GitHub's id for the user. No password.**

THEN, IDENTICAL IN BOTH `Auth::login($user)` → `session cookie` → `guard` → `policy`

The only difference is who checks the password. Every request after the login, and all of Part 3, is the same. (OAuth 2.0 can also grant access to data, such as a calendar; logging in is its most common use.)

OAuth 2.0: another service vouches for the user

DEFINITION OAuth 2.0 is a standard protocol (RFC 6749) that lets a user grant an application limited access to their account on another service, without giving that application their password.

THE FOUR ROLES, FOR "LOG IN WITH GITHUB" ON THE PANTRY

RESOURCE OWNER

Alex

Owns the GitHub account, and decides whether to say yes.

CLIENT

The pantry app

Wants Alex's name and email. Never sees the password.

AUTHORIZATION SERVER

GitHub's login

Checks the password, asks for consent, and issues tokens.

RESOURCE SERVER

GitHub's API

Gives the name and email to whoever shows a valid token.

Precisely: OAuth 2.0 grants access; by itself it does not say who the user is. For a login, your app uses that access to ask the provider "who is this?". OpenID Connect is the standard layer on top that adds this identity step. Why bother: your app never stores a password.

The authorization code flow: six steps, ending in a cookie

1 Redirect out *in the browser*

The pantry sends the browser to GitHub with its client_id, a redirect_uri and a random state.

2 Log in there *in the browser*

Alex types the password on GitHub's page and approves "Pantry wants your name and email".

3 Back with a code *in the browser*

GitHub redirects to the redirect_uri with a one-time code and the same state.

4 Code for a token *server to server*

The pantry's server sends the code and its client_secret. GitHub answers with an access token.

5 Ask who *server to server*

The pantry calls GitHub's API with the token and gets an id, a name and an email.

6 Your own session *yours*

Find or create the local user, then Auth::login(). A session cookie, as on the last slide.

PREDICT Why does step 3 carry a short-lived code, and not the access token itself?

Because URLs leak: browser history, server logs, referrer headers. The code works once, expires within minutes, and is useless without the client_secret, which only your server holds.

In Laravel: Socialite does steps 1 to 5; step 6 is yours

routes/web.php *the pattern from the Laravel Socialite docs (optional reading 8)*

```php
Route::get('/auth/redirect', function () {
    return Socialite::driver('github')->redirect();  // step 1
});

Route::get('/auth/callback', function () {
    $github = Socialite::driver('github')->user();   // 3 to 5
    $user = User::updateOrCreate(
        ['github_id' => $github->id],
        ['name' => $github->name, 'email' => $github->email]);
    Auth::login($user);                              // step 6
    return redirect('/');
});

config/services.php: client_id, client_secret, redirect
```

Step 2 has no line here: it happens on GitHub's page. Not needed for Homework 9; in week 13 the course platform becomes your provider, with these same two routes.

THREE LARAVEL PACKAGES, THREE ROLES

Socialite

Your app is the client: "Log in with GitHub / Google".

Passport

Your app is the authorization server: other apps log in with you.

Sanctum

Not OAuth. Session cookies for your own React app, plus simple API tokens. What the pantry uses.

A JWT is a signed statement the client carries

DEFINITION A JSON Web Token (RFC 7519) is a compact string of three base64url parts, header.payload.signature. It holds claims that the issuer has signed, so a receiver can verify them.

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
eyJzdWIiOiIxIiwibmFtZSI6IkFsZXggUml2ZXJhIiwicm9sZSI6InVzZXIiLCJleHAiOjE3OTQzMzAwMDB9.
1ERDEwIgx2Bj2BYkM-Ajm5fUarlLK9gT4Q7HpATfYxo
```

HEADER

```
{ "alg": "HS256",
  "typ": "JWT" }
```

Which algorithm signed it.

PAYLOAD: THE CLAIMS

```
{ "sub": "1", "name": "Alex Rivera",
  "role": "user", "exp": 1794330000 }
```

Who this is, and when the token expires.

SIGNATURE

```
HMAC-SHA256(
  header.payload, secret)
```

Proof that nobody changed the other two parts.

Signed is not encrypted. Base64url is an encoding, so anyone holding the token can read the payload. Never put a secret in a JWT.

The server verifies a JWT instead of looking it up

THE CLIENT SENDS

```
Authorization:
  Bearer eyJhbGciOi…
```

Your code adds the header. The browser does not do it for you.

THE API CHECKS

1. Recompute the signature.
2. Is exp still in the future?

Arithmetic and a clock. No database, no session store.

THE API TRUSTS

```
sub = 1, role = user
```

The claims stand in for the lookup. That is the gain, and the catch.

PREDICT Alex's token says "role": "user" and expires in an hour. The RA makes Alex an admin right now. What does Alex's next request claim? And what happens to the token when Alex logs out?

Still "user": *the claims are a snapshot from the moment of signing. And logging out cannot un-sign a token: it stays valid until exp, unless the server keeps a deny-list, which is a lookup again. That is why JWTs get short lifetimes.*

Session cookie or JWT? Ask who the client is

	Session cookie · a reference	JWT · a statement
What the client holds	a random ID that means nothing by itself	the claims themselves, signed
Where the login lives	on the server, in the session store	in the token
Who attaches it	the browser, automatically	your code, in the Authorization header
Log out, or revoke now	delete the session: immediate	hard: valid until exp
Fits	a browser app on your own site	APIs called by phones, scripts and other services
In Laravel	the session guard; Sanctum for a React app	Passport's access tokens; third-party packages

Rule of thumb: a browser app on your own origin gets a session cookie. That is your project. A JWT earns its place when the client is not a browser, or when many services must check the same login without sharing a session store.

Summary: two choices, three technologies

The choice	The baseline	The alternative
Who checks the user at login?	Your own password form. Your server compares a hash: Auth::attempt().	OAuth 2.0. Another service checks and vouches: Socialite.
What do later requests carry as proof?	A session cookie. A random ID the server looks up.	A JWT. Signed claims the server verifies, usually sent in a Bearer header.
In your project	Both, in Homework 9	OAuth 2.0 in week 13. No JWT.

Pick one from each row. Password + session cookie: Homework 9. OAuth 2.0 + session cookie: most "Log in with…" buttons, and week 13. Either login + a JWT: a phone app, or a public API.

OAuth 2.0 never replaces the cookie. It replaces the password form, and then your app starts its own session as usual.

Next: the server now knows who is asking. It has not yet decided what they may do.

PART 3 · AUTHORIZATION

We know who. What may they do?



Every technology in Part 2 ends the same way: the server knows who is asking. In Laravel, a guard supplies that user, and a policy makes the decision.

Three Laravel tools answer three different questions

TOOL	JOB	QUESTION	IN LARAVEL
Guard `auth:sanctum`	AUTHENTICATION	**Who are you?**	config/auth.php defines web (session cookie) and sanctum (API). Auth middleware supplies $request->user().
Gate `Gate::allows(…)`	AUTHORIZATION	**May you do this?**	The Gate facade is Laravel's authorization service for a single ability check.
Policy `TodoPolicy`	AUTHORIZATION	**May you do this to this kind of record?**	A class such as TodoPolicy groups model rules; Gate looks it up and calls the matching method.

Guard identifies the user. Gates and policies decide what that user may do.

Logged in is not the same as allowed

PREDICT Alex is logged in as a resident, so the React page hides the Delete button. Alex sends the request anyway. What should happen, and what happens if only the page knew the rule?

WHAT THE PAGE DOES · WEB/SRC/RULES.JS, SHORTENED

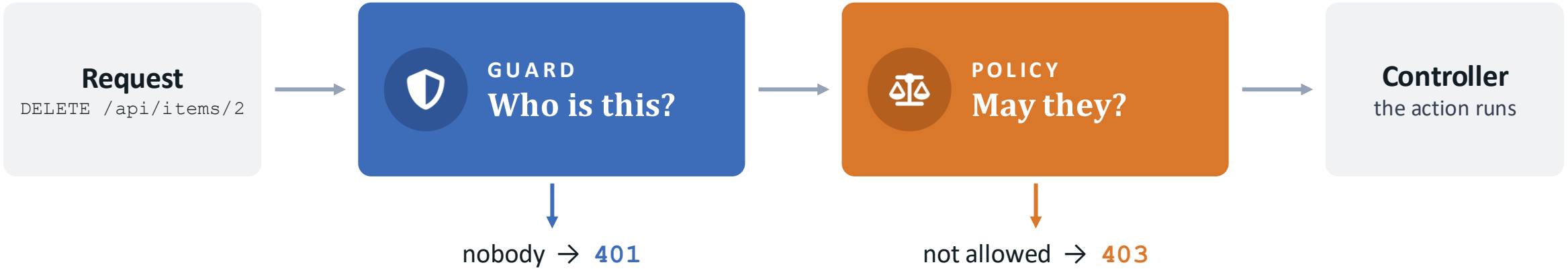
```
may this user do this? decides which buttons to draw
export function can(user, action, item = null) {
  if (!user) return false;
  if (user.role === 'admin') return true;
  if (action === 'update')
    return item?.user_id === user.id;
  return false;
}
```

WHAT ALEX DOES · THE BROWSER CONSOLE

```
no button needed: Alex writes the request by hand
const t = decodeURIComponent(
  document.cookie.match(/XSRF-TOKEN=([^;]+)/)[1]);
await fetch('/api/items/2', {
  method: 'DELETE',
  headers: { 'X-XSRF-TOKEN': t },
});
```

If only the page checked: `204`, and Priya's yogurt is gone. With the rule on the server: `403`. Hiding a button is good manners, not security: whoever sends the request decides what is in it. So the midterm's rules.js moves to the server, and the page keeps a copy only to decide what to draw.

Two checkpoints before the controller: guard, then policy



Guard

How Laravel authenticates each request: where to find the proof (a session cookie or a token) and how to turn it into a User.

`config/auth.php` · `auth:sanctum`

Gate

A closure that answers yes or no to one ability, such as "may this user see the list of users?"

`Gate::define(…)`

Policy

A class that groups the yes-or-no rules for one model, one method per action: create, update, delete.

`app/Policies/ItemPolicy.php`

A guard is the last step of authentication and the first input to authorization: a policy cannot decide about "nobody". Gates and policies are the two ways Laravel writes the rule.

A guard turns the cookie into $request->user()

```
config/auth.php: a guard is a driver plus a provider
'guards' => [
    'web' => ['driver' => 'session', 'provider' => 'users'],
],
'providers' => [
    'users' => ['driver' => 'eloquent', 'model' => User::class],
],

routes/api.php: which routes ask the guard
Route::get('/items', [ItemController::class, 'index']);     // anyone
Route::middleware('auth:sanctum')->group(function () {      // or 401
    Route::post('/items', [ItemController::class, 'store']);
    Route::put('/items/{item}', [ItemController::class, 'update']);
    Route::delete('/items/{item}', [ItemController::class, 'destroy']);
});
```

DRIVER AND PROVIDER

The driver says where the proof is: session means the session cookie. The provider says where users are: the User model.

AUTH:SANCTUM

Middleware that runs the guard. Sanctum tries the session cookie first, then a Bearer token.

THE RESULT

Nobody: 401, before your controller runs. Otherwise $request->user() is Alex, ready for the policy.

A policy: one class of yes-or-no answers for one model

app/Policies/ItemPolicy.php *pantry-example-final · found by name: Item → ItemPolicy*

```php
class ItemPolicy
{
    public function before(User $user): ?bool {
        return $user->isAdmin() ? true : null;    // the RA: yes
    }
    public function create(User $user): bool {
        return true;                              // any resident
    }
    public function update(User $user, Item $item): bool {
        return $item->user_id === $user->id;      // the owner
    }
    public function delete(User $user, Item $item): bool {
        return false;                             // RA only
    }
}
```

Make one with `php artisan make:policy ItemPolicy --model=Item`. From before(), null means "no answer here: ask the method".

THE RULE TABLE IT ENCODES

	Guest	Resident	Owner	RA
create	401	yes	yes	yes
update	401	403	yes	yes
delete	401	403	403	yes

WHY IT WORKS

Each method receives the logged-in User from the guard and the Item from the route, and returns true or false. A guest never gets this far: the guard has already answered 401.

IN ONE LINE

one model, one policy, one method per action

Ask the policy wherever a write happens

Three places to ask *1 and 2 are the pantry's code; 3 and the gate follow the Laravel docs*

```php
// 1. in a controller: ItemController::destroy()
Gate::authorize('delete', $item);      // no: 403, stops here
$item->delete();

// 2. in a form request: UpdateItemRequest::authorize()
return $this->user()->can('update', $this->route('item'));

// 3. on the route itself
Route::put('/items/{item}', [ItemController::class, 'update'])
    ->can('update', 'item');

// a gate: a rule with no model behind it
Gate::define('view-users', fn (User $u) => $u->isAdmin());
```

Green marks the line that asks the policy.

AUTHORIZE OR CAN

can() returns true or false, for when you want to branch. Gate::authorize() throws when the answer is no, and Laravel turns that into a 403 response.

WHY IT WORKS

All three call the same ItemPolicy method, so the rule is written once. The React page keeps its copy in rules.js only to hide buttons; if that copy is wrong, the API still refuses.

IN ONE LINE

guard → **401** · policy → **403**

Four requests. Predict the status code

PREDICT For each request, predict the status code, and name the checkpoint that answers: the guard, or which policy method.

1	**nobody: no cookie**	`POST /api/items`	a new item	?
2	**Alex, a resident**	`DELETE /api/items/2`	Priya's yogurt	?
3	**Alex, a resident**	`PUT /api/items/1`	Alex's own milk	?
4	**Jordan, the RA**	`DELETE /api/items/2`	Priya's yogurt	?

Answers: *1 → 401, the guard: no user, so the policy never runs. 2 → 403, ItemPolicy::delete() returns false. 3 → 200, update(): Alex owns item 1. 4 → 204, before() returns true for the RA. Same route as request 2, different user, different answer.*

Summary: one request, from cookie to controller

AUTHENTICATION · WHO

AUTHORIZATION · WHAT

Cookie

the browser attaches floor-pantry-session

→

Guard

auth:sanctum turns it into a User, or answers 401

→

Policy

ItemPolicy says yes, or the answer is 403

→

Controller

the action runs: 200, 201 or 204

PART 1

Two questions

Authentication: who are you? Fails with 401. Authorization: what may you do? Fails with 403. First who, then what.

PART 2

Three technologies

OAuth 2.0 can replace the password form. A JWT can replace the session ID. Your project: a password, then a session cookie.

PART 3

Two Laravel tools

A guard turns the proof into a User. A policy answers yes or no for one model. Ask it on every write.

The idea under all of it: the request belongs to a stranger. Prove who sent it, then decide on the server what they may do. The page only hides buttons.

IN-CLASS EXERCISES · STEP-THROUGH

The login drills, step by step



Eight drills on the to-do app from week 11, one for each idea from today.

For each: run the attack, make one fix, press Check result.

Set up: one folder, one port, two terminals

```
terminal 1: start the app, and leave it running
cd inclass/wk12/laravel
docker compose watch
wait for "Watch enabled", then open http://localhost:8020

terminal 2, same folder: put the tables back as they started
docker compose exec app php artisan migrate:fresh --seed

terminal 2: run a scripted attack (guess, delete or admin)
docker compose exec app php artisan attack guess
```

On the drill slides, `php artisan …` is short for `docker compose exec app php artisan …` in terminal 2. Port 8020 taken? `APP_PORT=8021 docker compose watch`.

THE SEEDED ACCOUNTS

Email	Password	Who
`alex@example.edu`	`password`	user, to-dos 1 to 4
`priya@example.edu`	`password`	user, to-dos 5 and 6
`admin@example.edu`	`admin-pass`	the admin

EVERY DRILL, THE SAME THREE MOVES

1. Run the attack, and watch it work.
2. Make the one change that closes it.
3. Press Check result on the drills page.

Eight drills, each one idea from today

Drill	The attack you run	The fix you make	From today
warm-up	Delete the session cookie in DevTools	none: you look	**Part 2 · cookies (done)**
1	/leak shows every password as typed	the hashed cast, and Hash::check	**Part 2 · the login**
2	Two error messages reveal which emails exist	one message for both failures	**Part 2 · the login**
3	A script guesses thirty passwords	a rate limit on POST /login	**Part 2 · the login**
4	A script deletes a to-do without logging in	move the route inside the auth group	**Part 3 · the guard**
5	Register with an extra role=admin field	create from validate(); role off Fillable	**Part 3 · the server decides**
6	Priya opens Alex's to-do by its number	query through the logged-in user	**Part 3 · logged in is not allowed**
7	Priya's search returns Alex's to-dos	search through the logged-in user	**Part 3 · logged in is not allowed**
8	A new rule: the admin may read, not change	TodoPolicy and Gate::authorize	**Part 3 · the policy**

Blue drills are about who you are; **orange** drills are about what you may do. Drills 1 to 3 and 5 add four defenses the lecture only named: do them to see why the login code has those lines.

Drill 1: passwords stored as typed

1 · RUN THE ATTACK

Open `http://localhost:8020/leak`.

It shows the users table as anyone with a copy of your database would see it. Every password is there, readable. A leaked table is a list of working logins.

3 · DONE WHEN

The check passes, and `/leak` shows `$2y$12$…` in every row.

EXPLAIN

Alex and Priya both chose "password". Are their two hashes the same? Why is that good?

2 · MAKE THE FIX *User.php, AuthController.php, then terminal 2*

```
app/Models/User.php, inside casts()
'password' => 'hashed',

AuthController::login(): compare with this, not with !==
Hash::check($credentials['password'], $user->password)

terminal 2: write the seeded rows again, hashed this time
php artisan migrate:fresh --seed
```

WHY IT WORKS

The hashed cast runs bcrypt on every save, so the table never holds the password itself. bcrypt is slow on purpose and salted, so a stolen table cannot be turned back into logins quickly. Hash::check hashes the typed password and compares: nothing is ever decrypted.

Drill 2: two error messages

1 · RUN THE ATTACK

Log in as `nobody@example.edu`.
Then log in as `alex@example.edu` with a wrong password.

Two different messages. The form just told a stranger which emails have accounts.

3 · DONE WHEN

The check passes: an unknown email and a wrong password now get the same message.

EXPLAIN

Why does Auth::attempt() make the two-message mistake hard to make?

2 · MAKE THE FIX *AuthController::login()*

```
replace the two if blocks with one
if (! $user || ! Hash::check(
        $credentials['password'], $user->password)) {
    return back()
        ->withErrors(['email' =>
            'These credentials do not match our records.'])
        ->onlyInput('email');
}
```

WHY IT WORKS

One message for both failures gives an attacker nothing to sort emails by. Laravel's Auth::attempt(), which the pantry uses, does the lookup and the password check in one call, so there is only one place to fail and only one message to write.

Drill 3: unlimited guesses

1 · RUN THE ATTACK

In terminal 2:
`php artisan attack guess`

Thirty common passwords against Priya's account. It finds hers on guess 25. The script fetched the login page first and sent a valid CSRF token: CSRF protection does not stop a script.

3 · DONE WHEN

The check passes, and the attack gets `429` on guess 6 and gives up.

EXPLAIN

Why is the limit keyed on the email and the address, and not on the email alone?

2 · MAKE THE FIX *AppServiceProvider.php, routes/web.php*

```
app/Providers/AppServiceProvider.php: un-comment in boot()
RateLimiter::for('login', fn (Request $request) =>
    Limit::perMinute(5)->by(
        strtolower((string) $request->input('email'))
        .'|'.$request->ip()));

routes/web.php
Route::post('/login', [AuthController::class, 'login'])
    ->middleware('throttle:login');
```

WHY IT WORKS

The limit counts tries for one email from one address: five a minute, then 429 Too Many Requests, before the controller runs. A million guesses now takes about 140 days.

Drill 4: a route outside the login

1 · RUN THE ATTACK

Log out. There is no Delete button anywhere: the pages only show it to someone logged in. In terminal 2:

```
php artisan attack delete 2
```

Alex's to-do 2 is gone, deleted by a script that never logged in.

3 · DONE WHEN

The check passes, and the attack is sent to `/login`.

EXPLAIN

Your team's check-login.sh asks this in check 7. What is an API's version of "sent to /login"?

2 · MAKE THE FIX *routes/web.php, then terminal 2*

```
move the delete route from the bottom into the group
Route::middleware('auth')->group(function () {
    Route::get('/todos', [TodoController::class, 'index']);
    …
    Route::delete('/todos/{id}',
        [TodoController::class, 'destroy']);
});

terminal 2: bring the to-do back
php artisan migrate:fresh --seed
```

WHY IT WORKS

auth is the guard, used as middleware: the first checkpoint from Part 3. It runs before the controller, so a visitor with no session never reaches destroy(). Hiding the button was never a check.

Drill 5: register yourself as an admin

1 · RUN THE ATTACK

Open `/register`. In DevTools, right-click the `<form>` → Edit as HTML, and add this before the button:
`<input name="role" value="admin">`

Register. You have an Admin link and every user's list. (Or: `php artisan attack admin`.)

3 · DONE WHEN

Both checks pass. The attack reports an ordinary user, and `/admin` answers 403.

EXPLAIN

Either fix alone stops this attack. Why do both?

2 · MAKE THE FIX *AuthController::register(), User.php*

```
AuthController::register(): create only what was validated
$fields = $request->validate([ … ]);
User::create($fields);

app/Models/User.php: an allow-list with no role in it
#[Fillable(['name', 'email', 'password'])]
class User extends Authenticatable
```

WHY IT WORKS

validate() returns only the fields it has rules for, so the extra role is dropped. Fillable is an allow-list of the columns a request may set, so role is ignored even if it gets that far. The client proposes; the server decides.

Drill 6: someone else's to-do, by its number

1 · RUN THE ATTACK

Log in as `priya@example.edu`. Her list has to-dos 5 and 6. Type this into the address bar: `http://localhost:8020/todos/2`

It is Alex's note. Press Mark done. Priya is logged in, so the guard let her through, and nothing asked whose to-do it is.

3 · DONE WHEN

The check passes, and `/todos/2` as Priya is a 404.

EXPLAIN

Why 404 here, and 403 in your team project? What is different about the data?

2 · MAKE THE FIX *TodoController::findTodo()*

```
before: any row in the whole table
after: only rows that belong to whoever is logged in
return $request->user()->todos()->findOrFail($id);
```

WHY IT WORKS

The query starts from the logged-in user, so another person's row is simply not found: 404. For private data that is the right answer, because it does not even confirm that to-do 2 exists. The pantry's items are public, so its policy answers 403 instead.

Drill 7: the search box

1 · RUN THE ATTACK

Still logged in as Priya, search for the letter `e`.

Alex's to-dos are in the results. The list was limited to the logged-in user; the search was not.

3 · DONE WHEN

The check passes: Priya's search returns only Priya's to-dos.

EXPLAIN

With fifty queries, how would you make sure none of them forgets?

2 · MAKE THE FIX *TodoController::index()*

```
start the search from the user, like the list
$todos = $request->user()->todos()
    ->when($q !== '', fn ($query) =>
        $query->where('title', 'like', "%$q%"))
    ->orderBy('title')
    ->get();
```

WHY IT WORKS

Drills 6 and 7 are one mistake in two places: a query that forgot to ask whose rows these are. On private data, every query has to start from the user.

Drill 8: a new rule that a query cannot express

1 · TRY IT (DO DRILL 6 FIRST)

Log in as `admin@example.edu` / `admin-pass`, open `/admin`, and click Alex's "Email the TA". 404: drill 6 says "only your own rows", and the admin owns none.

THE NEW RULE

The admin answers support requests, so the admin may read any to-do and delete one that breaks the rules. It is not the admin's list, so the admin may not mark it done.

PREDICT *Can you write this by changing the query in findTodo()?*

THE WHOLE RULE, AS A TABLE

Action	Route	The owner	The admin	Anyone else
`view`	`GET /todos/{id}`	yes	yes	404
`update`	`PATCH /todos/{id}/toggle`	yes	403	404
`delete`	`DELETE /todos/{id}`	yes	yes	404

THREE ANSWERS A POLICY METHOD CAN GIVE

The method returns	The browser gets	Use it when
`Response::allow()`	the page	the table says yes
`Response::deny('Why not.')`	403 Forbidden	they may know it exists, but may not do this
`Response::denyAsNotFound()`	404 Not Found	they should not learn that it exists

Answer: *no. view and update go through the same findTodo(), yet the admin gets a different answer for each. It depends on the action, not only on the row.*

Drill 8: write the rule once, in TodoPolicy

2 · YOUR TURN, TWO FILES

In TodoPolicy.php, every method starts as allow(). Write view() as shown, then update() and delete() from their rows of the table.

In findTodo(), look in the whole table again, then ask the policy.

3 · DONE WHEN

The drill 8 check passes and drill 6's still does. As the admin, /todos/2 opens, and Mark done answers 403.

EXPLAIN

The pantry's ItemPolicy uses before() to let the admin do anything. Why would before() be wrong here?

THE FIX *app/Policies/TodoPolicy.php, TodoController::findTodo()*

```
public function view(User $user, Todo $todo): Response {
    if ($todo->user_id === $user->id || $user->isAdmin()) {
        return Response::allow();
    }
    return Response::denyAsNotFound();      // anyone else: 404
}
public function update(User $user, Todo $todo): Response {
    if ($todo->user_id === $user->id) {
        return Response::allow();
    }
    return $user->isAdmin()
        ? Response::deny('Only the owner may change a to-do.')
        : Response::denyAsNotFound();
}
delete(): the same row of the table as view()

TodoController::findTodo(): find it anywhere, then ask
$todo = Todo::findOrFail($id);
Gate::authorize($ability, $todo);    // throws the 403 or 404
return $todo;
```

Run every check at once, and the errors you may meet

```
in the drills folder: every drill's check, in one run
docker compose run --rm test
```

What you see	Cause, and the fix
After drill 1, logging in gives a 500: "does not use the Bcrypt algorithm"	The rows were written before the cast. Run php artisan migrate:fresh --seed.
attack guess answers 429 from the very first guess	The limit works, and is still counting your last run. Wait a minute.
attack delete says "There is no to-do 2"	An earlier attack deleted it. Run php artisan migrate:fresh --seed.
Drill 8: the admin still gets 404 on /todos/2	findTodo() still starts from the user. Drill 8 replaces drill 6's line.
Drill 8: Priya gets 403, and drill 6 now fails	The policy returned deny() for her. Anyone else gets denyAsNotFound().
Drill 8: everyone may do everything	Gate::authorize($ability, $todo) is missing, so the policy is never asked.

More in the Troubleshooting table at the end of inclass/wk12/laravel/README.md. Worked answers for every drill are in the answers/ folder: try each drill before you look.

Friday: the same ideas on your own api/, in nine files

THE API · FILES 1 TO 5

1. **`User.php`**
   the hashed cast; role stays off Fillable
2. **`AuthController.php`**
   register, login, logout, user
3. **`routes/api.php`**
   the throttled login; the auth:sanctum group
4. **`AppServiceProvider.php`**
   the two RateLimiter::for() calls
5. **`bootstrap/app.php`**
   statefulApi(), and your web port in compose

THE PAGES · FILES 6 TO 9

6. **`web/src/api.js`**
   currentUser, login, register, logout
7. **`web/src/session.jsx`**
   SessionProvider and useSession()
8. **`pages/AuthPages.jsx`**
   the Log in and Register forms
9. **`Your layout`**
   the user's name and a Log out button

```
from your team repo, with docker compose watch running
bash ../itws2110-course/inclass/wk12/check-login.sh http://localhost:8010
the goal for the API side:  9 passed, 0 failed
```

Every file exists in project/pantry-example-final under the same path: read the example's version, then write yours. Full steps: inclass/wk12/README.md.

Friday's lab and Homework 9 use every part of today

1	**The login**	POST /register, /login, /logout and GET /user on your api/. Seeded users can log in.	Part 2 · the session cookie
2	**The write endpoints**	The routes that answered 501 in Homework 7 now work, inside the auth:sanctum group.	Part 3 · the guard
3	**One rule**	A policy for your resource: who may edit, who may delete. 401 for a guest, 403 for the wrong user.	Part 3 · the policy
4	**The pages**	Log in, register and log out in React. The buttons the midterm hid stay hidden; the server now enforces it.	Part 1 · both questions

Friday 11/13: the login lab, in class, on your own api/, with a check script. Homework 9 is a team assignment, due Fri 11/20; the full page , with exactly what is graded, goes up Friday. Everything on this slide is already in project/pantry-example-final.

TWO QUESTIONS, ON EVERY REQUEST

Who are you?
What may you do?



Authentication proves it once, then carries the proof on every request.
Authorization decides on the server, every time. Friday, you build both.